# Föreläsning 5
# Programutvecklingsteknik

- Abstrakta klasser
- Dynamisk bindning
- Polymorphism
- Klassen Object
- Designmönstret Observer
- Omslagsklasser

# Innehåll

- Abstrakta klasser
- Dynamisk bindning
- Polymorphism
- Klassen Object
- Designmönstret Observer
- Omslagsklasser

# Abstrakt klass

- Definieras med nyckelorden `abstract`
  - T.ex `abstract class Fordon{…}`
- En **abstrakt klass** kan inte instansieras direkt
  - `new Fordon();` ger kompileringsfel
- Minst en **abstrakt metod** för att klassen fylla upp sin *mening, abstrakt*
  - `abstract void kör(int hastighet);`
- Används för att definiera en **gemensam bas** för flera subklasser
- Ger möjlighet att **blanda konkret och abstrakt funktionalitet**
- Konkret subklass måste kokretisera abstrakta metoder
- Abstrakt subklass *kan men måste inte* kokretisera abstrakta metoder
- Vi går igenom filerna `AbstraktKlass.java` och `Schackpjas.java`

# Bättre designprinciper

- **Hög sammanhållning**
  - En klass eller modul bör ha ett tydligt enda ansvar
  - Bidrar till återanvändbarhet (klassen kan användas i andra sammanhang/program)
- **Lös koppling**
  - Betyder att objekt **inte känner till varandras exakta implementationer**.
  - De kommunicerar via **abstraktioner**, t.ex. **gränssnitt** eller **abstrakta klasser**.
  - Fördelen är att man kan **ändra eller byta ut** en klass utan att påverka de som använder den.
  - Vi går igenom filen `LooseCoupling.java`

# Dynamisk bindning

- Vilken metod som anropas bestäms **under körning**, inte vid kompilering
- Möjliggör **polymorft beteende** – rätt metod i rätt objekt används
- Bygger på att **anrop sker via referenser till superklasser eller gränssnitt**.
- Viktig skillnad: **statisk bindning** gäller t.ex. för statiska metoder och privata metoder.
- Vi går igenom filen `DynamicVsStatic.java`

# Polymorphism

- Betyder "många former" – samma metodanrop kan ge olika beteenden.
- Gör koden **flexibel och utbyggbar** utan att ändra befintlig logik.
- Kräver ofta **arv eller gränssnitt**.
- Tillsammans med dynamisk bindning ger det **riktig objektorientering**.
  - Det är viktigt att skilja mellan Dynamisk bindning och Polymorfism.
  - Dynamisk bindning är en mekanism som alltid finns där och det möjliggör att man kan påstadkomma polymorfism.
  - Alltså i en programkod behöver inte polymorfism implementeras trots att mekanismen Dynamisk bindning finns där.
  - Mekanismen Dynamisk bindning kan inte väljas bort, finnas alltid där.

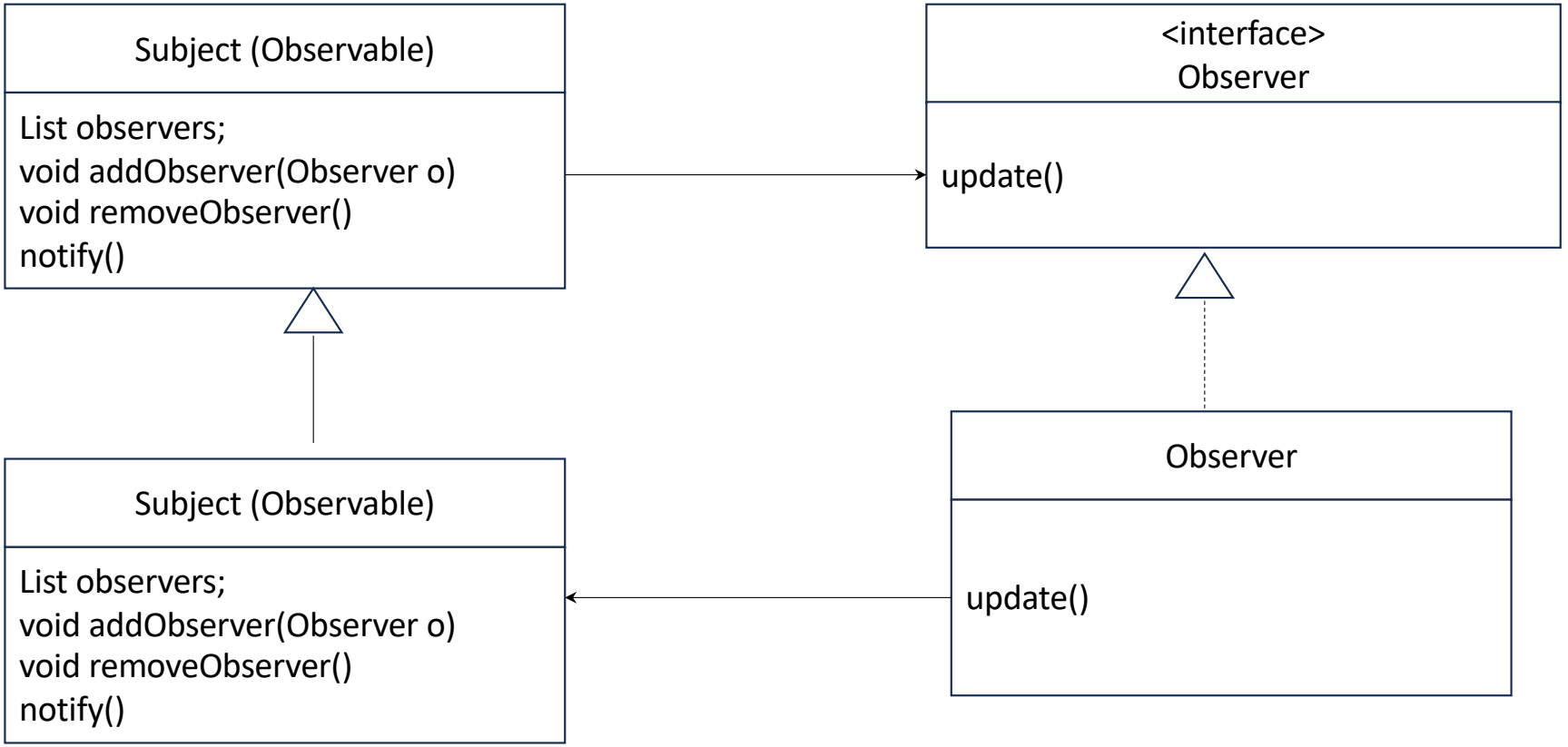
# Klassen Object

- Alla klasser i Java **ärver indirekt från** klassen `Object`.
- Alla klasser ärver **grundläggande metoder** som:
  - `toString(), equals(),…`
- Dessa metoder **kan (och bör)** överskuggas vid behov
- Möjliggör **generell hantering av objekt**, t.ex. i samlingar `(Collections, ArrayList,List,…)`
- **Överskuggning** (override) är när man definierar om en metod som finns i superklassen
- **Överlagring** (overload) är när man definierar om en metod i samma klass, förstås med olika parametrar I antal eller typ

# Designmönstret Observer

- **Problem:** Ett objekt behöver informera intressenter när dess tillstånd ändras. Direkt koppling mellan objektet och intressenter leder till inflexibilitet, hård och svårunderhållen kod.
- **Lösning:** Handlar om **kommunikation mellan objekt** – ett "notifieringsmönster".
- En **subject (observable)** meddelar en ellera flera **observers** när något ändras.
- Ger **lös koppling** mellan avsändare och mottagare.
- Vi går igenom votexemplet

# Designmönstret Observer



# Omslagsklasser

- Ger objektrepresentation av **primitiva datatyper**.
- Exempel: `Integer, Double, Boolean, Character, …`
- Möjliggör att **lagra primitiva värden I samlingar** (List<Integer>).
- Stöd för **autoboxing/unboxing**: automatisk konvertering mellan typ och omslagsklass.